

A Real-Time 2D/3D Perception Visual Vector Processor for 1920×1080 High-Resolution High-Speed Intelligent Vision Chips

Siyuan Wei^{ID}, Ke Ning^{ID}, Lei Kang^{ID}, Xuemin Zheng^{ID}, Mingxin Zhao^{ID}, Mengmeng Xu^{ID}, Shuyu Wang, Xuanzhe Xu, Runjiang Dou^{ID}, *Member, IEEE*, Shuangming Yu^{ID}, Xu Yang^{ID}, *Member, IEEE*, Jian Liu^{ID}, Cong Shi^{ID}, *Member, IEEE*, Nanjian Wu, *Senior Member, IEEE*, and Liyuan Liu^{ID}, *Member, IEEE*

Abstract—Edge computing of reliable multimodal (2D RGB/3D RGB-Depth) data has a wide range of applications. However, many of currently reported visual processors cannot flexibly handle multimodal data, e.g., the visual streams of RGB-Depth data. The key challenge exists that these prior visual processors do not come with efficient and unified instruction set architecture (ISA) for both conventional and intelligent cognition on the 2D/3D multimodal sensory data. To fill such a gap, this paper proposes a programmable intelligent visual vector processor compatible with multimodal 2D/3D visual data processing (1920×1080 -pixel resolution). The processor consists of a reconfigurable processing element (PE) array, a memory access network flexibly configurable to be fine- or coarse-grained, and a high throughput I/O interface. The vectorial PE array with neighbor PE access increases the data reuse rate and parallel computation efficiency, and can implement both convolutional neural networks (CNNs) and conventional image processing algorithms. The proposed ISA is customized and optimally tailored targeting 2D/3D image processing from RGB/Time-of-Flight(ToF)

raw data to intelligent inference results. The chip is fabricated in a 55-nm CMOS process. The experimental results showed that the area efficiency, peak performance, and peak throughput of our chip attained as high as 14.41 GOPS/mm², 409.6 GOPS, and 9.6 Gbps at 200 MHz, respectively. The measured processing speeds of this chip on ToF depth reconstruction is 87 fps (480×270) or 31 fps (1920×1080), on 3D object classification is 219 fps (256×256), and on CNN-based 2D object tracking is 36 fps (256×256).

Index Terms—Visual processor, near-sensor computing, parallel computation, 2D/3D image processing, vision chip.

I. INTRODUCTION

INSPIRED by the structural organization of the human visual system that tightly combines the eye retina and the visual cortex, the vision chip [1] integrates an image sensor and massively parallel computing elements on a single silicon substrate, to form a compact real-time visual system-on-chip (SoC), suitable for end-to-end cognitive visual perception in edge scenarios. Nowadays, current reported vision chips achieve intelligent edge processing with the fast development of deep neural networks (DNN) [2]. Simultaneously, the increasing image resolution and imaging modes, including 2D gray imaging and 3D depth imaging, can improve the capability and intelligent level of vision chips. However, it requires vision chips to have a high throughput and flexible processing capability to handle this massive and multi-type visual information.

To increase the processing rate of captured images, vision chips integrate single-instruction multiple-data (SIMD) processing elements (PE) arrays. Early vision chips embed one PE into each pixel, thus achieving pixel-level parallel processing [3], [4] for ultra-low latency. But this paradigm incurs large area overhead, low fill factor, and low pixel resolution, leading to degraded image quality against subsequent robust processing. Moreover, such fixed pixel-PE correspondence limits the function of those vision chips to pre-processing, such as image filtering, edge enhancement, and thresholding.

To solve this problem, some vision chips integrate a sensor and a processor in separate regions and connect them via a high bandwidth data interface [5], [6], [7], [8], [9], [10], [11]. In this manner, the PE array processor can have a flexible mapping relationship with the pixels, so that the

Manuscript received 5 March 2023; revised 26 September 2023; accepted 29 November 2023. Date of publication 15 December 2023; date of current version 30 January 2024. This work was supported in part by the National Key Research and Development Program of China under Grant 2019YFB2204303; in part by the National Natural Science Foundation of China under Grant U20A20205 and Grant 62274154; in part by the Beijing Municipal Science and Technology Project under Grant Z221100007722028; in part by the Beijing Natural Science Foundation under Grant Z2200005; in part by the Youth Innovation Promotion Association Program Chinese Academy of Sciences under Grant 2021109; in part by the Open Research Funding from the State Key Laboratory of Computer Architecture, Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), under Grant CARCH201908; and in part by the Chongqing Natural Science Foundation (Post-Doctoral Foundation) under Grant cstc2021jcyj-bsh0126. This article was recommended by Associate Editor P. Girard. (*Corresponding author: Liyuan Liu.*)

Siyuan Wei, Ke Ning, Lei Kang, Xuemin Zheng, Mingxin Zhao, Mengmeng Xu, Shuyu Wang, Xuanzhe Xu, Runjiang Dou, Shuangming Yu, Xu Yang, Jian Liu, and Liyuan Liu are with the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences, Beijing 100083, China, and also with the Center of Materials Science and Optoelectronics Engineering, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: liuly@semi.ac.cn).

Cong Shi is with the School of Microelectronics and Communication Engineering, Chongqing University, Chongqing 400044, China.

Nanjian Wu is with the State Key Laboratory of Superlattices and Microstructures, the Center for Excellence in Brain Science and Intelligence Technology, and the Institute of Semiconductors, Chinese Academy of Sciences, Beijing 100083, China, and also with the Center of Materials Science and Optoelectronics Engineering, University of Chinese Academy of Sciences, Beijing 100049, China.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSI.2023.3339784>.

Digital Object Identifier 10.1109/TCSI.2023.3339784

sensor pixel array can be scaled up independently without adding the same number of PEs, and the sensed image quality is improved while the total chip area remains reasonable. More importantly, such separated sensor-processor architecture allows incorporating advanced processors to support various conventional CV tasks, such as row-parallel processor array [6] for tracking based on mathematical morphology, and dynamic reconfigurable PE array [10], [11] for self-organizing map neural networks. However, when DNNs become state-of-the-art in the many domains of intelligent image processing, the processors of these vision chips cannot efficiently process the huge amount of multiply-accumulate (MAC) operations in DNNs.

To process DNN efficiently, intelligent vision chips with parallel MAC units are optimized in the aspects of PE architecture, data reuse, and data transmission [12], [13], [14], [15], [16]. The design in [13] adopts hybrid data type MAC units in PE to replace multipliers with shifters and adders, improving energy efficiency. A 3D stacked vision chip [15] proposed integrates 2304 MAC units and an efficient tensor data memory access mechanism to run DNN models for visual classification and detection tasks at a high speed (30fps, 4056×3040). However, the improvement of high-performance DNN models on vision chips is costly. A DNN model contains millions of parameters. Given such consideration, conventional CV methods are still necessary for vision chips to handle some vision tasks. For example, the Lucas-Kanade model [17] is a widely used CV method for fast optical flow estimation. To efficiently support both conventional CV models and DNN models, the vision chips in [18] and [19] employ multi-functional PEs together with dynamically reconfigurable connections among vectorial PE groups. Nevertheless, those reported vision chips have all focused on 2D image (gray-scale or RGB-color) processing.

Compared with 2D images, 3D images introduce the concept of depth (D) into the RGB-D image structure. 3D image processing techniques become significant in both industrial and recreational fields such as autonomous driving [20], robotics [21], and augmented reality [22]. Nowadays, image sensors have developed rapidly [23], the works in [24], [25], and [26] have already realized multimodal 2D/3D image capture on a single chip. However, existing vision chips lack the functionality for such multimodal 2D/3D image processing, which poses a big challenge for the next-generation vision chip design. As the main computing unit of the prior vision chips, their PEs lack flexible instruction set architecture (ISA) to implement both conventional 2D/3D CV and convolutional neural network (CNN).

To foster fast and compact multimodal vision chips, this work proposes a real-time 2D/3D perception visual vector processor dedicated to conventional 2D/3D CV tasks and CNNs, including 2D visual object classification/detection as well as 3D image depth reconstruction. First, we propose a custom instruction set architecture (ISA) that supports flexible SIMD operations to implement 2D/3D image processing on the visual processor. The proposed processor supports flexible parallel PE micro-operations (like reduced instruction-set computer operation). Our PEs merge the function of image

signal processors (ISP) and CNN processors to improve area efficiency. Next, we propose a reconfigurable PE array that realizes a data reuse pattern through local routing. Local data in a PE can be reused by neighbor PEs. By reconfiguring the PEs as different PE groups, the PE groups make full use of the PE array.

The main contributions of this work are the following:

(1) We design a visual vector processor for high-speed vision chips to implement 2D/3D CV. The processor integrates a reconfigurable programmable PE array for high flexibility. Each PE integrates hybrid computing units to support massive mathematical and logic operations for 2D/3D image processing. PE array realizes complex functions to eliminate redundant computing components, so the processor achieves a comparably high area efficiency of 14.41 GOPS/mm² with high flexibility.

(2) We design a 2D/3D CV-specific ISA for the PE array. A mapping scheme is designed to implement multimodal image processing algorithms based on the custom ISA, by applying optimized data flow to make full use of the PE array. Most instructions are vectorial for high parallelism.

(3) We optimize data access of the PE array motivated by data reuse and data loading. For frequent data reuse operations, we implement a data share mechanism among neighbor PEs. For frequent data loading operations, the PE array implements a dual-issue micro-structure to simultaneously process data movement with computation.

(4) The chip is fabricated in a 55 nm process. A general-vision-purpose parallel architecture is built to test the processor. Several 2D/3D image processing algorithms have been deployed on this chip and can be real-time (> 30 fps) processed.

This paper is organized as follows. The visual processor architecture is presented in Section II. The essential circuit of this visual processor is described in Section III. In Section IV, we demonstrate the mapping scheme of 2D/3D image processing. Experimental results, comparisons, and discussions are shown in Section V. Finally, conclusions are provided in Section VI.

II. SYSTEM ARCHITECTURE

Fig. 1 illustrates the overall architecture of the visual vector processor we proposed. This work comprises a programmable PE array for high-performance computing, a programmable microprocessor unit (MPU) for global control, a multi-level data memory that supports flexible memory access, a high-speed I/O interface, and the image transformation circuit (consist of data reorganization, image sample, and dynamic range adjustment modules). The PE array constitutes a heterogeneous vision processing unit (VPU) with a scalar core. The scalar core is a micro control unit to manage instruction schedule flow and on-chip data memory access. To support massive mathematical and logic operations for 2D/3D image processing, each PE integrates hybrid computing units and general-purpose register files. Multi-functional PEs effectively reduce redundant processing circuits to achieve high area efficiency. PE array receives decoded instructions from the scalar core and executes vectorial instructions. The total 256 PEs

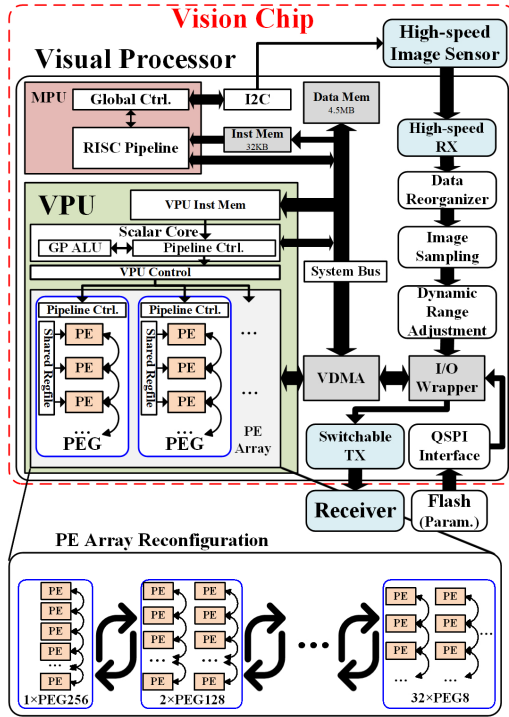


Fig. 1. The system architecture of the visual vector processor.

are divided into variable-scale PE groups (PEGs) to compute or access data memory parallelly. PEs within a PEG form a vector block and can share operands to increase data reuse rate or load data respectively to increase data throughput. The dimension of the vector changes within 8/16/32/64/128/256 through PEG reconfiguration. MPU is a RISC-based micro control unit. As the global controller of this visual processor, MPU controls the VPU program counter, off-chip data transmission, image sampling configuration, sensor configuration, I/O interface switching, and conflict monitor. MPU is programmed through C language and globally schedules the task of VPU. The instructions of MPU and VPU are independent and receive 32-bit instructions from their instruction memory. The VPU data memory access (VDMA) comprises fine-grained and coarse-grained memory access networks with data memory. Fine-grained memory access means a small segment of total PEs or the scalar core load/store from on-chip data memory, coarse-grained memory access means multiple PEs load/store from on-chip data memory. RX interface of this processor receives standard MIPI protocol data from a 1920×1080 high-speed image sensor. RGB/gray/Time-of-Flight (TOF) raw data is recovered to full image or sampled image through the image transformation circuit. TX interface is used to transmit received images from image sensors or processed results from data memory. A PLL is used to generate core clock.

The processing flow of this visual processor is described below. First, MPU instructions and VPU instructions are loaded into instruction memory after the chip reset. Then, MPU starts to execute MPU instructions and takes control of the processor. MPU instructions include sensor configuration (through I2C), image transmission mode configuration, image import, algorithm parameter import, start of the VPU, state

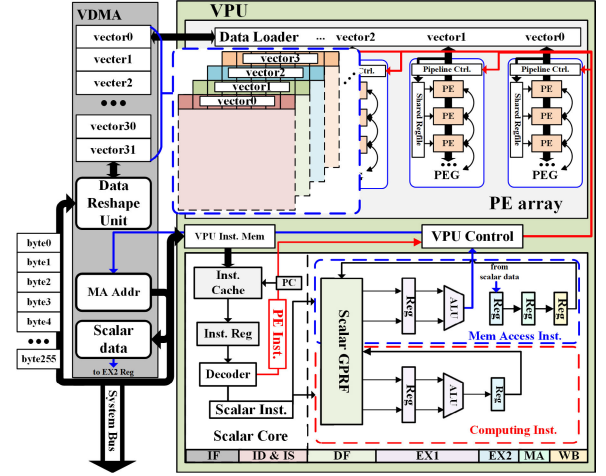


Fig. 2. The architecture of the VPU and VDMA. The VPU consists of a reconfigurable PE array and scalar core. The scalar core executes scalar instructions locally or issues decoded vector instructions to the VPU controller. The red lines indicates the data path of computing instructions and the blue lines indicates the data path of memory access instructions.

monitor, data export, and computing operations. For convolutional neural network (CNN) inference, the VPU loads weight and input feature map through the VDMA from on-chip data memory to local register files. The VPU implements an output stationary data flow, which means that the partial sum of the convolution result is statically accumulated in local PE register files, but the input feature maps and weights are dynamically loaded from data memory or neighbor PEs.

III. CIRCUITS DESIGN

This section starts with an introduction to the essential circuits of this work. These essential circuits are designed for flexible and efficient CV implementation.

A. VPU Architecture

The architecture of VPU and VDMA is shown in Fig. 2. PE array provides main computing capacity, it comprises PEGs, data sharing register files (SRF), and multi-level data loaders. VPU instruction pipeline comprises 7 segments, including instruction fetch (IF), instruction decode and instruction issue (ID&IS), data fetch (DF), execution step1 (EX1), execution step2 (EX2), memory access (MA), and write back (WB). Scalar core and PE array execute instructions separately but share the first two segments of the pipeline circuit. Vector instructions are executed by the PE array for vectorial parallel processing, scalar instructions are executed by the scalar core for serial processing. Scalar core broadcasts decoded instruction codes to the PE array, all PEs execute the same operations to calculate vector-scale operands. Compared with a typical 5-stage CPU pipeline, VP has a DF step to adopt large-scale data access of VDMA (vectorial data load/store) and an extra execution step EX2 to adopt complex ALU functions (such as MACs). This pipeline extension also reduces the maximum delay of each segment to increase the frequency of system clock. Though VPU CNN implementation uses an output stationary data flow, feature map transmission between on-chip memory and PE register files is still frequent. Therefore,

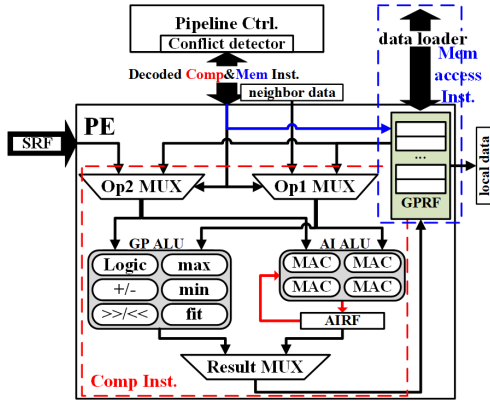


Fig. 3. PE architecture and corresponding computing units. Hybrid function ALUs support massive operations for 2D/3D CV implementation. Components used for computing operations are marked in red, and components used for memory access operations are marked in blue.

we implement an instruction dual-issue micro-architecture [19] into PEs and scalar core to issue memory access instructions and computing instructions simultaneously. Alternate storage of these two types of instruction can increase the VPU instruction throughput while executing memory access and computing operations at one time. The results of computing instruction can be written back to the register file directly to avoid the cost of further memory access. The optimization rate of dual-issue micro-architecture depends on the proportion of memory access instructions. Implementations with more memory access and high data reuse rate instructions are able to achieve better optimization.

B. PE Architecture and PE Array

The architecture of a single PE is depicted in Fig. 3, which shows the components and data path of operands. Each PE has independent register files and computing units to execute pixel-level processing tasks independently. For conventional CV, each PE implements a general-purpose ALU (GP ALU) for mathematical operations and logic operations. In particular, we add fitting operations to perform piecewise linear functions to fit non-linear activation functions (Sigmoid, Tanh). Linear fitting is a fast and low-cost method to obtain an approximation of complex functions. For intelligent CV (CNNs), each PE implements an artificial intelligence (AI) ALU for MAC operations. Considering the high parallelism of CNN inference, 4 MAC units are integrated into each AI ALU to accelerate CNN inference and share the control circuit. Those hybrid functional ALUs are designed to support general 2D/3D CV implementation.

The bit-width of computing units is important to precision loss and computational consumption. Compared to 32-bit float-point inference, using 8-bit weights and 32-bit activations for inference only results in an accuracy drop of 0.56% on the MobileNetV2 [27]. To balance the inference precision and hardware complexity of our design, operands of PE arithmetic and logic units (ALU) are designed as 8-bit fixed-point for common instructions and 16-bit fixed-point for high precision instructions, and 32-bit fixed-point for activation results.

PE includes a 16×16 -bit general-purpose register file (GPRF) to store parameters (such as weights, input feature

maps, and inference results), and a 4×32 -bit AI-purpose register file (AIRF) to store activations and high precision results. PEG8 is the fundamental unit executing computing instructions, comprised of 8 PEs, SRF, data loader, and pipeline controller. All 8 PEs in a PEG8 can access data from SRF to reduce redundant data load instructions. Typical 2D convolutions in a CNN model have a high data reuse rate for both input feature maps and weight. Either feature map or weight needs to be broadcast through the register files of several PEs. We design a data path for all the PEs to acquire data from neighbor PEs inside a PEG8 to increase the data reuse rate. All the PEs from PE0 to PE255 are connected to 2 adjacent PEs from the left and right sides. Therefore, each PE is connected to 4 neighbor PEs. CNN weight is stored in SRF and the partial sum is stored in AIRF. During the process of partial sum accumulation, the input feature map needs to be transferred among PEGs, the partial sum is accumulated in local PE, and CNN weight is shared in SRF.

To accelerate highly-parallel algorithms, large PEGs can be dynamically reconfigured by PEG8s through VPU instructions. The total 32 PEG8s can reconfigure as multi-scale blocks (PEG16, PEG32, PEG64, PEG128 and PEG256). Large PEG blocks are dedicated to large feature map computing or high parallelism computing. As shown in Fig. 4, a total 32 PEG8s can be reconfigured as one PEG256 to process $c_{in} \times 256 \times N$ (c_{in} is the number of input channels, N is the height of input feature map) feature map or reconfigured as 32 PEG8s to process $c_{in} \times 8 \times N$ feature map. The reconfiguration is realized by local routing. The routing (controlled by the operand multiplexer) in each PE is configured by the state registers in the pipeline controller, the state registers are configured through PE reconfigure instructions. When configured as a different PE group (PEG), the connection of PEs at the edge of the PE group is cut off from the PE outside the PE group. Those edge PEs obtain padding data when fetched outside the PEG. This change of data path is designed to support window sliding of CNNs and filters. Zero padding and edge copy are supported by border PEs inside a PEGX. Reconfiguration of PEG improves the flexibility and utilization of PEs when computing feature maps of different sizes. The dual-issue micro-architecture of PE has been discussed. The components of memory access and computing instruction are mostly separate to avoid conflict.

C. VPU Instruction Set

A CV-specific vectorial ISA is designed for VPU. This VPU instruction set ensures that conventional and intelligent algorithms are able to be deployed on this visual processor. Instruction-level programmability is supported by this 32-bit SIMD reduced instruction set. The instruction type of VPU is described in Table I. In particular, vector-type instruction contains extra operation code to perform conditional operations. These three types of vector instructions comprise different segments of the pipeline. Basic computing operations comprise IF, ID&IS, DF, EX1, and WB. AI instructions comprise IF, ID&IS, DF, EX1, and EX2, MAC is finished at EX1 and EX2; MAC results are stored in AIRF so MA and WB are omitted. Memory access instructions comprise the total of 7 segments.

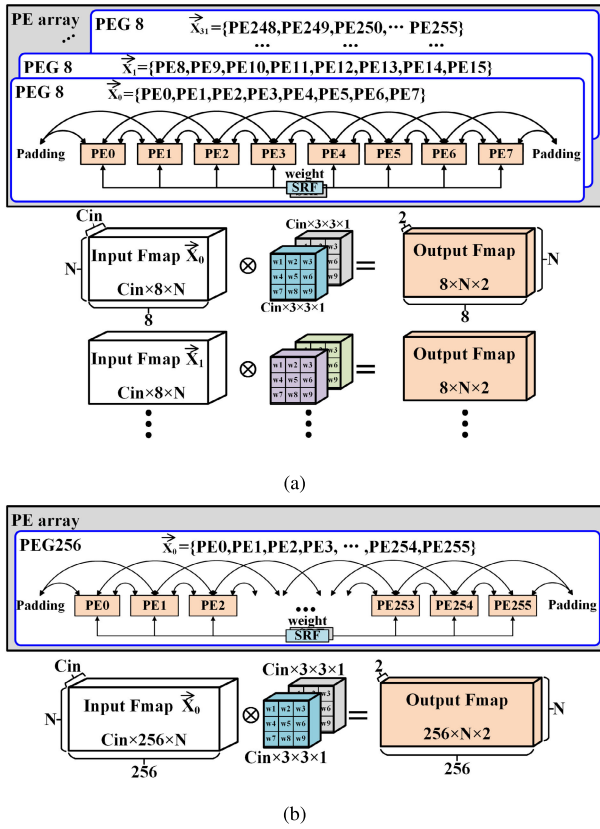


Fig. 4. PEG reconfiguration for different parallelism. Variable vector dimension is supported through PEG reconfiguration. (a) PEG8 reconfiguration. (b) PEG256 reconfiguration.

VPU instruction memory supports a maximum instruction amount of 16384, which is enough to store VPU instructions of a 100-layer CNN model. Vector computing instructions include general computing, MAC computing, linear fitting, and memory access.

Scalar instructions comprise on-chip memory access, computing, branch&jump. The operands of scalar instructions come from the scalar core register file (SCRF). These scalar instructions schedule the data flow of VPU. The processing flow of convolution with conditional jump executed by VPU is shown in Algorithm.1 by pseudo-code. Typically, a processing flow starts with memory address acquisition. The memory address is computed by the scalar core with a base address and an offset, the offset can be an immediate operand or an operand from a SCRF. Next, the vector-type memory access instruction and MAC instructions are executed. Furthermore, jump&branch instructions can be used to construct if/else and for/while control flow. The processing flow of other functions (such as activation function and approximate function) can be rewritten based on Algorithm.1.

A part of the PEs might do idle work when computing small-scale scalar data. Integrating scalar core and PE array in one computing unit and sharing an instruction set makes the processor compatible with both serial and parallel instructions. Instruction scheduling is mainly controlled by jump&branch instructions while the conditional operation of computing instructions can also control write-back operation to reduce program complexity. To compute a $k_h \times k_w$ kernel with an

TABLE I
VPU INSTRUCTION SET

Inst Type	Operation Code	Description
Scalar Type	Basic Operation	16b logic, mathematical
	Memory access	SCRF load/store
	Jump & branch	Conditional or unconditional
Vector Type	Basic Operation	8b/16b logic, mathematical, function fitting
	Memory Access	GPRF/SRF load/store
	AI Operations	MAC with neighbor PE data access
Ctrl. Type	PE Reconfigure	Change vector dimension

input feature map, all PEs run their MACs for $k_h \times k_w$ times and get 256 (width) $\times$ 2 (height) convolution results of N output channels. When these computing instructions circularly execute for c_{out}/N times, the computation of a $256 \times N$ output feature map from all output channels is complete. Similarly, a loop implementation is applied to the CNN mapping of VPU instructions. The complete data flow and mapping scheme of CNN and conventional algorithms are described in section IV.

D. Multi-Level Data Memory Access Networks

The memory access networks are designed to improve data load/store efficiency with variable vector dimensions and routing modes (such as unicast, broadcast, and multicast). PE Data rearrangement and down sampling are common methods for image processing on this visual processor. We design variable vector dimension memory access networks for PEG. Each PEG8 contains a low-level data loader for PE-level data access, each PEG64 contains a middle-level data loader for block-level data access and the PE array contains a high-level data

Algorithm 1 Pseudo-Code for Simplified Convolution

```

1: function MAC(gprf1, gprf2, srfl, neighboraccess)
2:   [AIR0; AIR1; AIR2; AIR3] = AIRF
3:   AIR0 = AIR0 + gprf1  $\times$  srfl[7 : 0]
4:   AIR1 = AIR1 + gprf2  $\times$  srfl[7 : 0]
5:   AIR2 = AIR2 + gprf1  $\times$  srfl[15 : 0]
6:   AIR3 = AIR3 + gprf2  $\times$  srfl[15 : 0]
7:   AIRF = [AIR0; AIR1; AIR2; AIR3]  $\triangleright$  MAC
   function is a single instruction of the VPU, operands can
   be fetched from gprf of neighbor PEs or the local PE
8: end function
9:   SCR1 = 256  $\triangleright$  set jump flag
10:  SCR8 = ADDRfeaturemap  $\triangleright$  set input address
11:  SCR9 = ADDRweight  $\triangleright$  set weight address
12:  loop1:  $\triangleright$  set jump destination
13:    SRF1 = DataMem[SCR9]  $\triangleright$  load weight
14:    GPRF1 = DataMem[SCR8]  $\triangleright$  load input
15:    GPRF4 = DataMem[SCR8+256]  $\triangleright$  load new input
16:    MAC(GPRF1,GPRF4,SRF1,1)  $\triangleright$  operands from the left
17:    MAC(GPRF1,GPRF4,SRF1)  $\triangleright$  local operands
18:    MAC(GPRF1,GPRF4,SRF1,r1)  $\triangleright$  operands from the
    right
19:    SCR1 = SCR1-1
20:  if SCR1 > 0 then  $\triangleright$  jump judgement
21:    jump back to loop1
22: end if

```

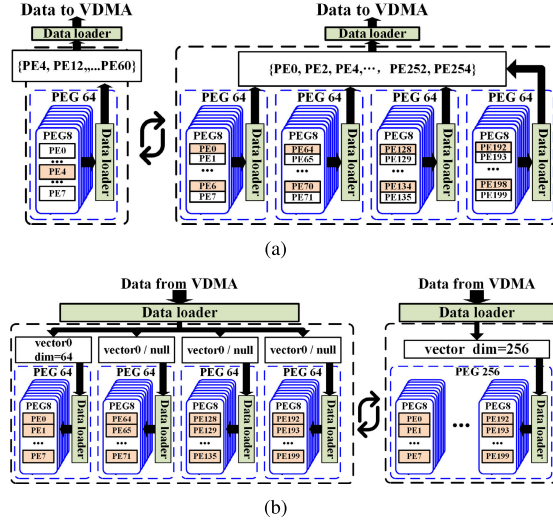


Fig. 5. Data load/store from multi-level data memory access networks after PEG reconfiguration. The reconfiguration of PEG changes the data store/load pattern to achieve high flexibility. (a) Fine & coarse-grained store pattern of PEG. (b) Fine & coarse-grained load pattern of PEG.

loader for array-level data access. From the data storage aspect, a fine-grained store is able to store from a single PE in a PEG8, coarse-grained store is able to store from multiple PEs in a PEG8. From the data loading aspect, each PEG receives original or copied data from the memory access networks. The dimension of the loaded vector data matches the reconfigured PEG dimension. Fig.5 demonstrates multi-level load/store patterns of data memory access networks. Fig.5 (a) shows two of the supported data store patterns. The first is the data memory access networks store the data to VDMA from 1/8 PEs of a PEG64. The second is to store data from half the PEs. The pattern is selected by instructions. Fig.5 (b) shows two of the supported data load patterns. The first is the data memory access networks load the data from VDMA to a PEG64 and choose whether to broadcast this vectorial data to the remaining PEG64s. The second is to load data to all the PEs. The pattern is selected by instructions. The VPU data memory is the on-chip data memory for input images, algorithm parameters, intermediate results, and inference results.

E. MPU

MPU is a RISC-based micro-control unit designed for off-chip memory access and global control. The 32-bit RISC instruction set of MPU realizes integer operation, memory access, branch & jump. Internal state monitor is realized by state registers and is applied to control the VPU and I/O interface by reading or writing specific address space on the system bus. Write or read operations are transferred to state operations through state monitors of registers. Input/output task, sensor configuration, and VPU state monitor are programmed in C language. We use a standard ISA-based GCC compiler to compile the C language program. The MPU and VPU can execute instructions simultaneously.

IV. ALGORITHM MAPPING

This section starts with an introduction to a flexible instruction-level 2D/3D compatible algorithm mapping

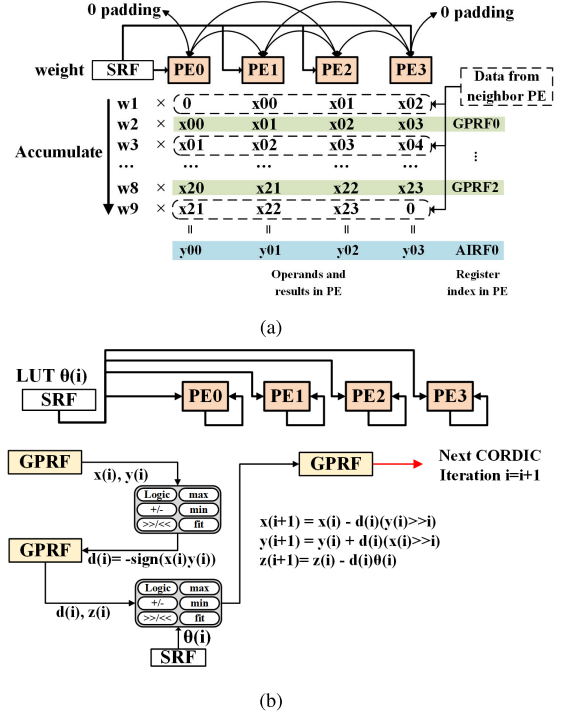


Fig. 6. PE workflow of 2D convolution and general operation. (a) 2D convolution workflow on PE. (b) CORDIC iterative workflow on PE.

scheme. We apply this scheme to CNNs and conventional CV tasks to maximize the efficiency of the corresponding circuit design described in the above sections.

A. PE Workflow in Implementation

CV implementation on PE has two types of typical workflows, a general workflow for int8/int16 independent pixel-level computing, and a convolutional workflow for int8/int16 vector-level computing. As shown in Fig.6 (a), CNN workflow contains frequent data sharing among PEs and SRF. The workflow contains continuous MAC operations. The partial sum in AIRF is calculated by accumulating the dot product between the input feature map in the GPRF and the weights in the SRF. Fig.6 (b) shows the CORDIC iterative workflow as an example of general workflow, which is an essential approximation algorithm for hardware to implement trigonometric functions. The further application of CORDIC is mentioned in subsection E. This general workflow contains iterative PE-level ALU operations.

B. CNN Mapping

The CNN mapping method makes full use of PEG reconfigurability and neighbor data access. The inference of CNN layer and CNN variations is widespread in the field of artificial intelligence. For a typical CNN layer, a $c_{in} \times H \times W$ input feature map (c_{in} is the number of input channel, H and W indicate height and width) needs a $c_{in} \times k_h \times k_w \times c_{out}$ weight tensor to calculate an $H \times W \times c_{out}$ output feature map with padding. Equation.1 shows the computing of convolution, Y is the output feature map, W is the weight tensor, X_{pad} is the input feature map with padding, and s is

stride.

$$Y(c_{out}, h, w) = \sum_{c_{in}, k_h, k_w} W(c_{in}, k_h, k_w, c_{out}) \times X_{pad}(c_{in}, h_s + k_h, w_s + k_w) + bias(c_{out}) \quad (1)$$

Consider the parallelism of CNN. An input feature map X can be unfolded as $c_{in} \times H$ input vectors with a dimension of W . Tensor $X(c)$ is a channel of input feature map X as Equation.2 shows.

$$\begin{aligned} X(c) &= \{\vec{X}_c(1), \vec{X}_c(2), \dots, \vec{X}_c(H)\} \\ \vec{X}_c(h) &= [X_{pad}(c, h, 1), X_{pad}(c, h, 2), \dots, X_{pad}(c, h, W)] \\ X &= \{X(1), X(2), \dots, X(c_{in})\} \end{aligned} \quad (2)$$

Considering the padding elements of tensor $X(c)$, $X_{pad}(c)$ can be written as Equation.3.

$$X_{pad}(c) = \{\vec{0}, [0, \vec{X}_c(1), 0], [0, \vec{X}_c(2), 0], \dots, [0, \vec{X}_c(H), 0], \vec{0}\} \quad (3)$$

With neighbor PE data access, all the W PEs can load data from left or right PE. $X_c(h)_l$ is the data loaded from left, $X_c(h)_r$ is the data loaded from right. The vectors loaded from neighbor PEs are shown in Equation.4 and 5:

$$X_c(h)_l = [0, X_{pad}(c, h, 1), X_{pad}(c, h, 2), \dots, X_{pad}(c, h, W - 1)] \quad (4)$$

$$X_c(h)_r = [X_{pad}(c, h, 2), X_{pad}(c, h, 3), \dots, X_{pad}(c, h, W), 0] \quad (5)$$

The convolution of $X_c(h)$ with a $c_{in} \times 3 \times 3 \times c_{out}$ weight tensor can be transferred to vector form in Equation.6:

$$\begin{aligned} \vec{Y}_{c_{out}}(h) &= \sum_{c_{in}, k_h} W(c_{in}, k_h, 1, c_{out}) \times \vec{X}_c(h + k_h - 2)_l \\ &+ W(c_{in}, k_h, 2, c_{out}) \times \vec{X}_c(h + k_h - 2) \\ &+ W(c_{in}, k_h, 3, c_{out}) \times \vec{X}_c(h + k_h - 2)_r + bias(c) \end{aligned} \quad (6)$$

Considering the partial sum calculation process of one input channel, a parallel block is constructed with $1 \times 4 \times W$ input vectors (1-input channel, 4-height, W -width) and $1 \times k_h \times k_w \times 2$ kernels (1-input channel, k_h -kernel height, k_w -kernel width, 2-output channel). Each parallel block can obtain a $2 \times W \times 2$ (2-height, W -width, 2-output channel) partial sum. Fig.7 shows the input and output of a parallel block. This Parallel block is the basic computing task executed by a PEG8. The output feature map is obtained by accumulating the output results of parallel blocks of all the input channels. The filter sliding inside a parallel block is realized by neighbor PE data access and input vector dimension alteration is realized by PEG reconfiguration. A typical CNN layer needs $c_{in} \times H \times W \times k_h \times k_w \times c_{out}$ multiplication operations. With parallelism of 1024 through 1024 MAC units in 256 PEs, the parallel block mentioned above can be computed within $k_h \times k_w$ cycles. Each $k_h \times k_w$ kernels are reused for $k_h \times k_w$ times. Accumulating parallel blocks in the direction of the input channel can obtain 2 rows of the W -dimensional output feature map. A 3-loops implementation can be applied to

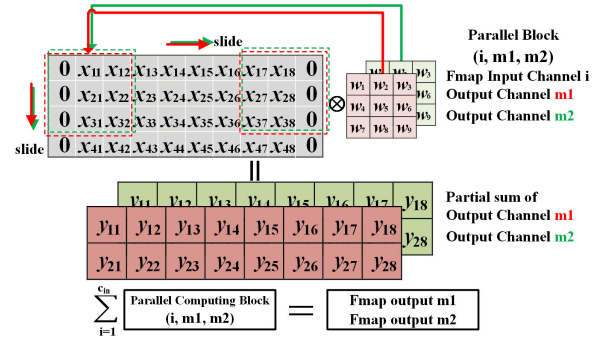


Fig. 7. Decompose CNN inference by parallel blocks. Accumulate the results of parallel blocks to obtain an output feature map. A parallel block can compute the partial sum of 2 output channels simultaneously. 8-D input vector and $1 \times 3 \times 2$ kernels are input data of a parallel blocks. Output channel m1 and output channel m2 are processed parallelly within a block.

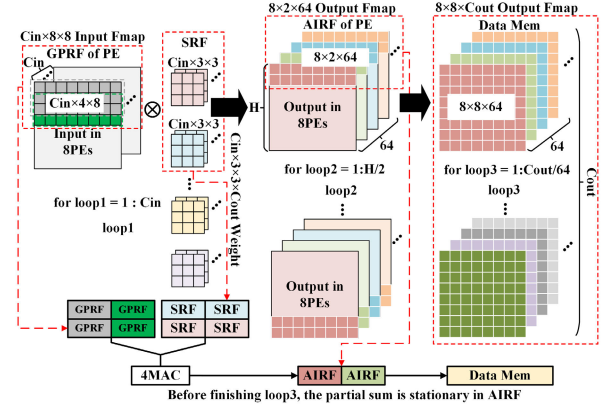


Fig. 8. 8-D vector CNN computing flow in PE arrays.

this vector decomposition for CNN inference as Fig.8 shows. Loop1 is on the input channel (C_{in}) of the input feature maps to accumulate the partial sum of all input channels. Loop2 is on the height (H) of the input feature map. Loop3 is on the output channel (C_{out}) of the output feature maps to finish the computation of all output channels.

C. CNN Variation Mapping

Variations of CNN have been proposed to construct lightweight DNN models. MobileNet V1 [28] includes point-wise (PW) and depth-wise (DW) convolution to reduce the parameter amount of the model. Similarly, a parallel computing block of DW convolution can be constructed with $1 \times 4 \times W$ input vectors (W -dimensional) and $k_h \times k_w \times 2$ kernels. The partial sum of the input channel does not need to be accumulated, so the total processing flow only needs loop2 and loop3 compared with the CNN data flow described in subsection B.

PW convolution is a special form of convolution with a $c_{in} \times 1 \times 1 \times c_{out}$ weight tensor. The parallel block of PW convolution is similar to the normal CNN layers.

D. Fully-Connected Layer Mapping

Fully-connected (FC) layer is a variation of matrix multiplication. An FC layer can be described as:

$$Y(c_{out}, h, w) = \sum_{c, h, w} W(c, h, w, c_{out}) \times x(c, h, w) + bias(c_{out}) \quad (7)$$

TABLE II
CORDIC LUT

Iteration(i)	$\tan \theta_i$	θ_i
0	1	45.0
1	2^{-1}	26.565
2	2^{-2}	14.036
3	2^{-3}	7.125
...	...	...

A $c_{in} \times H \times W$ input feature map is convoluted by a $c_{in} \times H \times W \times c_{out}$ weight tensor to obtain a $c_{out} \times H \times W$ output feature map. The parallel block of the FC layer is similar to the CNN layer, only if the weight of the FC layer cannot be reused.

E. Conventional CV Mapping

PE has already supported massive mathematical operations for conventional CV, therefore we focus on the operation that is not supported by the ALU of PE. For example, the transcendental function is widely used in CV implementation, such as ToF image depth reconstruction. The formula of the ToF indirect principle (light source modulated sinusoidally) [29] is shown in Equation.8, c_0 to c_3 are the sensor data from the same pixel of different phases, c is the velocity of light, f is the modulation frequency of the sensor. The formula includes transcendental function, which is suitable to approximately compute by CORDIC algorithm on hardware.

$$L = \frac{c}{4\pi f} \arctan\left(\frac{c_3 - c_1}{c_0 - c_2}\right) \quad (8)$$

CORDIC algorithm is a look-up-table (LUT) based iterative method to approximately calculate transcendental functions, such as $\sin/\cos/\arctan$. CORDIC is hardware-friendly to get a approximate result of transcendental function with basic operations like addition and shift. The CORDIC iteration formulas are listed below and LUT to calculate the $\arctan$ function is shown in Table.II. Initial approximate value of z_0 is 0, the angle accumulate iteration to approximately calculate $\arctan(\frac{y_0}{x_0})$ is shown in Equation.9, k is the rotate factor, i is the number of iteration.

$$\begin{aligned} d_i &= -\text{sign}(x_i y_i), \quad x_{i+1} = x_i - d_i (2^{-i} y_i) \\ y_{i+1} &= y_i + d_i (2^{-i} x_i), \quad z_{i+1} = z_i - d_i \theta_i \\ k_n &= \prod_n (\sqrt{1 + 2^{-2i}}), \\ n \rightarrow \infty : x_n &\rightarrow k_n \sqrt{x_0^2 + y_0^2}, \quad y_n \rightarrow 0 \\ z_n &\rightarrow z_0 + \arctan \frac{y_0}{x_0} \end{aligned} \quad (9)$$

The LUT of the rotation angle can be loaded to hardware data memory before the start of the iteration.

F. Algorithm Implementation and Simulation

Software verification and RTL (designed by System Verilog) hardware simulation are used to evaluate the algorithm. 8-bit quantized models are widely used in edge devices with low computing consumption and low precision loss. The fixed-point parameters of the DNN model we use are quantized by

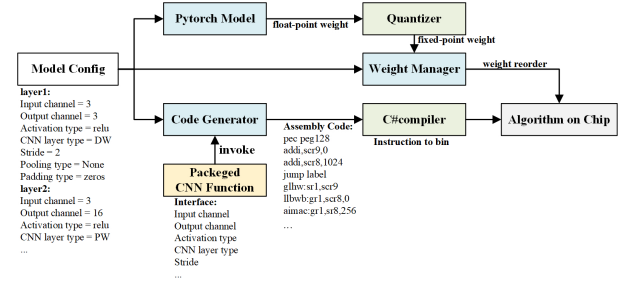


Fig. 9. Mapping phase from the target algorithm to the hardware.

a post-training quantization method in [30]. We used fixed-point tools in [31] and [32] to simulate the complete chip processing flow and evaluate the performance of fully fixed-point inference. Fig. 9 shows the workflow of the mapping from the Pytorch model to the hardware model. The mapping flow of this work has two steps. The first step is to generate the target CNN model written by assembly code (instructions mentioned in Table I). The second step is to transfer code written by assembly code to binary 32-bit instructions. As we mentioned in Algorithm 1, the pseudo-code of convolution shows the outline of a convolutional layer written by assembly code. The actual usage of instructions to construct a convolutional layer is determined by the parameters of the layer. For example, a PE reconfiguration instruction is needed at the start of a layer and the size of PEGs to be configured is determined by the width of the input feature map. Therefore, we packaged some common functions that would be used in CNN. We write a code generator in Python to invoke the functions we package to generate assembly code. The code generator receives the parameter of layers in CNN. The second step is finished by a C# compiler to transfer instructions written by assembly code to 32-bit binary instructions. The RTL model has been implemented on an FPGA hardware system simulator to verify the design of this visual processor.

V. EXPERIMENTAL RESULTS AND COMPARISONS

A. Chip Fabrication

The proposed processor was fabricated in a 55 nm CMOS process with a die size of 7.198 mm $\times$ 8.098 mm, the die photo is shown in Fig.10. It can receive 9.6Gbps RGB/gray/TOF raw data input from high-speed image sensor, 12bit per pixel at a core clock frequency of 200MHz. The specification of this chip is listed in Table.III. The peak performance of this visual processor is 409.6 GOPS with 256 PEs and 1024 MAC units. The chip has a 10MHz QSPI interface (40Mbps) for parameter and instruction loading, and a 200kHz I2C interface for sensor configuration. 4.5MB data memory (internal SRAM) was fabricated for image, feature map, and parameter storage. Large on-chip memory avoids frequent off-chip memory access for parameter/feature map transmission during data processing. This visual processor can generate 16-bit fixed-point depth images at 31 fps (1920 $\times$ 1080, error of CORDIC $< 0.89^\circ$ compared to $\arctan$ function) using ToF 3D image based indirect method, at 12.3 fps (1920 $\times$ 1080) or 87 fps (480 $\times$ 270) (error of CORDIC $< 0.022^\circ$ compared to $\arctan$ function). 4 AI algorithms are programmed on this work

TABLE III
CHIP SPECIFICATION

Fabrication Process	55 nm
Chip size	7.198 mm(H) $\times$ 8.098 mm(W)
Image input type	RGB/Gray/TOF
MAX resolution of sensor	1080(H) $\times$ 1920(W)
Supply voltage	1.08 V-1.3 V Core / 2.5 V Pad / 1.2 V Analog
Input Reference clock	40 MHz
Core clock	200 MHz
Interface	0.8 Gbps $\times$ 12lane MIPI
	QSPI 40 Mbps
	I2C
	Parallel interface 640 Mbps
Power	MIPI TX on 356 mW at 1.08 V
	MIPI TX off 268 mW at 1.08 V
Number of PEs	256
Total Power ratio	Core 68.54%
	MIPI TX 25.23%
	MIPI RX 6.23%
	Clock network 64.47%
Core Power ratio	Register 21.55%
	Memory 9.17%
	Combinational 2.71%
	I/O pad 2.10%
	Core 47.8%
	MIPI TX 3.0%
Area ratio	MIPI RX 3.9%
	PLL 0.5%
	Memory 39.1%
	Other part 12%

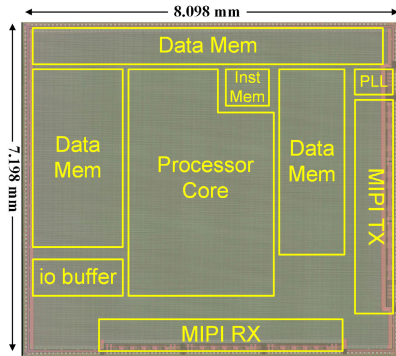


Fig. 10. Die photo of 55 nm visual processor.

with the proposed instruction set, processing latency < 30 ms and peak power < 0.36 W.

B. Evaluation

We deployed several 2D/3D image pre-processing and intelligent processing algorithms on the proposed visual processor. These algorithms achieved a frame rate > 30 fps. Fig. 11 shows the evaluation system. An FPGA evaluation board (Xilinx VCU118 platform) is used to configure the chip, generate input images (MIPI CSI-2) and receive output results from the visual processor. Algorithm parameters, instructions and images are loaded to on-board DDR and transferred to the corresponding protocol by FPGA. FPGA MIPI DPHY IPs are used to generate input images to simulate the image capture process of the sensor. Vision algorithms for 2D/3D image processing were implemented on this visual processor by programming to verify the correctness of the chip operations and evaluate the performance of the fabricated visual processor. With the dual-issue architecture, the total throughput of the FC layer and

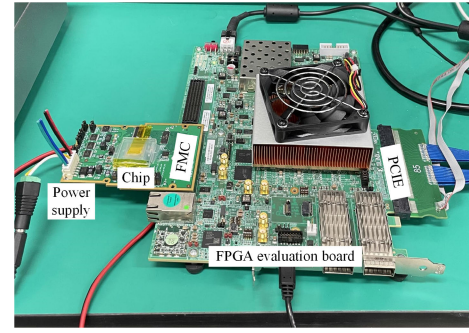


Fig. 11. Visual processor evaluation system. FPGA evaluation board provides image input and algorithm parameter input.

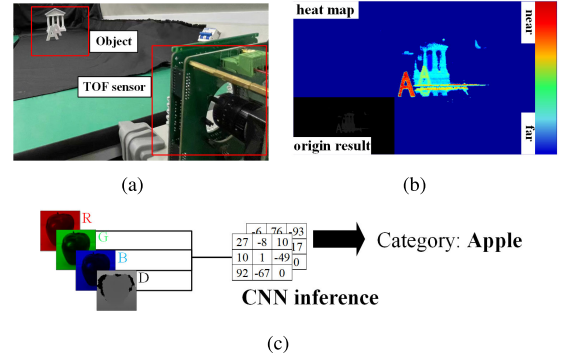


Fig. 12. ToF depth reconstruction and multimodal CNN inference. (a) Data capture scene. (b) Depth reconstruction result and heat map. (c) RGB-D CNN classification.

CNN layer computation increases 16% to 68% compared with serial execution. For CNNs, the measured power consumption is 268 mW, and the energy efficiency is 1.49 TOPS/W (8-bit).

1) *Depth Map Pre-Processing and RGBD CNN Classification*: Depth reconstruction is a pixel-level pre-processing algorithm for ToF data as formula (9) described, using the CORDIC algorithm to reconstruct a depth map. We implemented depth reconstruction with resolutions of 480×270 , 960×540 , 1920×1080 on the proposed visual processor. The processing speed of 480×270 ToF image reconstruction on the visual processor reached 87 fps with a power consumption of 219 mW and energy efficiency of 234 GOPS/W (16-bit). Four ToF images of phase 0° , phase 90° , phase 180° and phase 270° are required for a depth map. With the 3D information of objects, we applied a demo classification CNN on RGB-D images (219 fps, 8-bit). The 5-layer demo CNN is trained through an RGB-D object dataset [34] and achieved an accuracy of 97% on the evaluation set. The inference result shows the proposed visual processor is able to process 2D/3D multimodal data. Fig. 12 shows the environment of ToF image capture, the heat map of the depth reconstruction result, and the demo RGB-D algorithm. Furthermore, the reconstructed RGB-D image can be transferred to point clouds for other applications by a projector calibration method in [33].

2) *RGB Image Pre-Processing Algorithm*: RGB image pre-processing algorithm can be applied before intelligent algorithms to improve the quality of raw data from the sensor. We implemented Gaussian blur and brightness enhancement as examples to verify the correctness of the visual processor. Gaussian blur and brightness enhancement are effective pre-processing algorithms for 2D image processing. Compared to

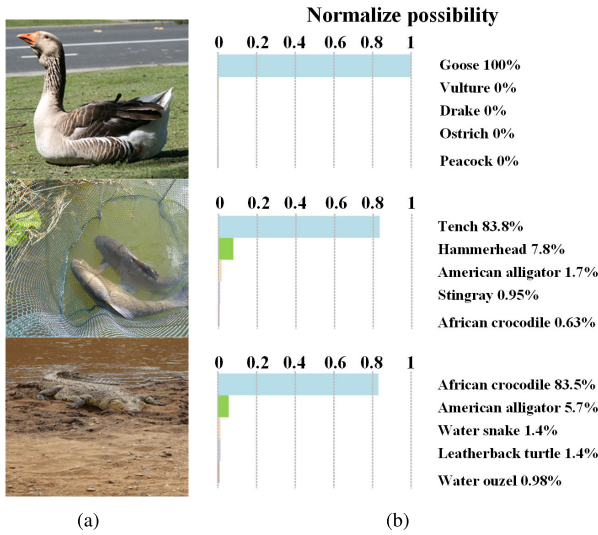


Fig. 13. MobileNet inference results of ImageNet. (a) Input images. (b) Inference results on the chip.

CNN and Depth map reconstruction, the latency of these basic algorithms is negligible ($< 10 \mu s$). The measured power consumption is 278 mW and the energy efficiency is 719 GOPS/W (8-bit).

3) *Image Classification CNN Inference*: MobileNet is a typical lightweight CNN classification algorithm [28]. We design a 28-layer MobileNet-based CNN classification network. This MobileNet-based CNN consists of convolution, DW, PW, pooling, and FC layers. After post-training quantization, the int8 quantized model achieved an accuracy of 89.8%(top-5) on a 100-class subset of the ImageNet dataset [35] with a Bayer input image size of 512×448 . Bayer image is transferred to RGB $256 \times 224 \times 3$ image through image transformation circuits. Quantized algorithm parameters are reduced to 2.9 MB in total. The test system achieves 31 fps with an input image frame rate of 70 fps from FPGA MIPI DPHY IP. The processing speed of the visual processor reaches 48 fps. FPGA throughput is the bottleneck of this demo system. Fig.13 shows the inference result.

4) *Face Detection CNN Inference*: SSD [36] is a one-stage object detection algorithm with the advantage of low latency compared with other CNN object detection models. We design a 67-layer MobileNet & SSD-based CNN for face detection. This CNN model comprises convolution, DW, PW, residual block, and SSD detection head. The offset of default anchors from the detection head is decoded by matrix calculation to get the position and confidence of objects. After post-training quantization, the int8 quantized MobileNet-SSD achieved a mean average precision (mAP) of 49.8% (IOU = 0.5) on a single face subset of the WiderFace dataset [37] with a Bayer-pattern input image (512×512). Bayer image is transferred to RGB $256 \times 256 \times 3$ image through image transformation circuits. Quantized model parameters were reduced to 960 kB in total. The test system achieved 37 fps with input images from FPGA. The processing speed of the visual processor reached 98 fps. Fig.14 shows the inference result and model structure.

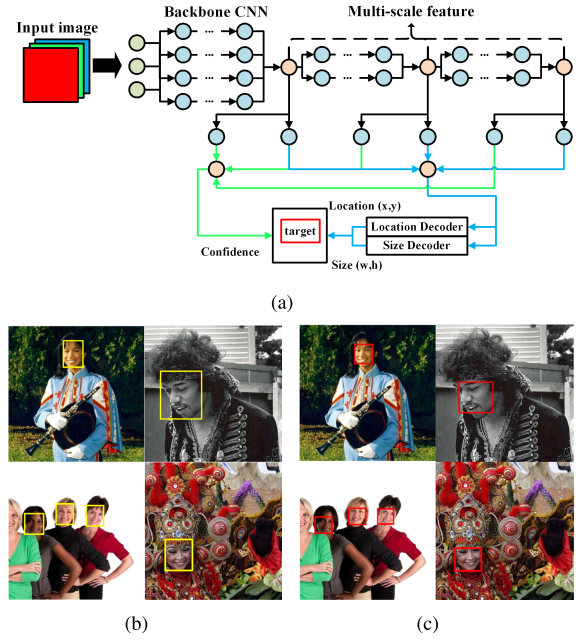


Fig. 14. MobileNet SSD face detection results on the WiderFace dataset. (a) MobileNet-SSD structure. (b) Ground truth. (c) Inference results on the chip.

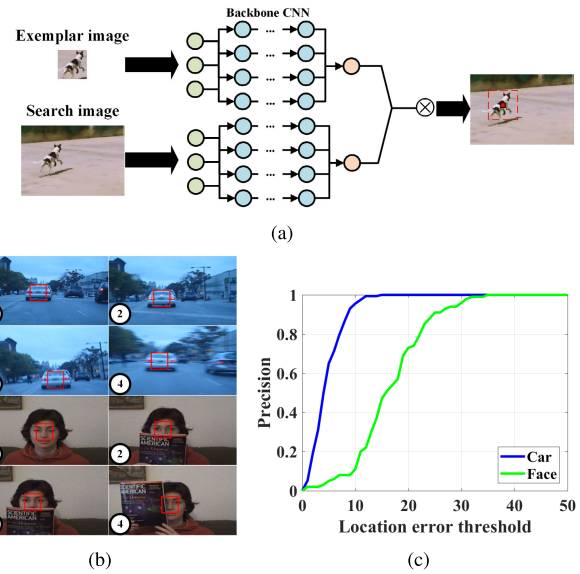


Fig. 15. SiamFC-Mobile tracking results on human face and car subset from the OTB100. (a) SiamFC-Mobile structure. (b) The inference results on the chip. (c) The precision plot of inference results on the chip.

5) *Target Tracking CNN Inference*: SiamFC is a fully-convolutional object tracking algorithm focusing on learning strong embeddings in an offline phase [38]. The structure of SiamFC is extremely simple and appropriate for edge devices. The feature map of the background and target is extracted by an identical CNN model. To capture the position of the target in the background, a cross-correlation layer is used to calculate the correlation between the background and target feature map. We design a 16-layer SiamFC & MobileNet-based neural network for tracking. Original SiamFC has 11×11 kernels to achieve a large receptive field. To increase algorithm processing efficiency, we replaced 11×11 kernels with two 5×5 kernels. This quantized algorithm tracks a 128×128 target from a 256×256 background. The background

TABLE IV
PERFORMANCE COMPARISON

	ISSCC2017 [8]	JSSC2018 [13]	JSSC2019 [14]	ISSCC2021 [15]	VLSI2022* ¹ [40]	This work
Process(nm CMOS)	40	65	130	22	22	55
Area (mm ²)	25.51	16	187	62.0	8.76	58.29
Clock Frequency (MHz)	108	100	80	262.5	190	200
On-chip SRAM	187 kB	N/A	171 kB	9 MB	1428 kB	4.5 MB
Main-computing components	4-b PE + 1-b PE	Detector + PE	PE	ISP + PE	PE	PE
Data Type(int)	1, 4b	16b	8b	8, 16, 32b	8, 16b	8, 16b
Supply Voltage (V)	N/A	0.46-0.8	1.2	0.8	0.5-1.0	1.08-1.3
Max Throughput (Gbps)	3.46	N/A	1.2	8.4	N/A	9.6
Imager Resolution(H/W)	1296×976	320×240	1024×768	4056×3040	Not support	1920×1080
Peak Performance (8b)	140 GOPS (4b)	N/A	61 GOPS	1.21 TOPS	511 GOPS	409.6 GOPS
Energy Efficiency (8b)	N/A	1.2 TOPS/W* ⁸ 0.22 TOPS/W* ² (16b)	85 GOPS/W 105 GOPS/W* ²	2.05 TOPS/W* ⁶ 1.12 TOPS/W* ²	12.1 TOPS/W* ⁷ 2.59 TOPS/W ²	1.49 TOPS/W* ²
Power(mW)* ⁹	32 (PE only)	211	720	379.1	158	268* ⁴
Area Efficiency (GOPS/mm ² 8b)	5.49 (4b) 2.90* ³ (4b)	N/A	0.33 1.82* ³	19.51 3.12* ³	58.33 9.33 * ³	7.06 14.41 * ⁵
Algorithms Support	Conventional CV	CNN	Conventional CV	CNN	Robot Navigation	CNNs and Conventional CV
Imager Data Modal	2D RGB	2D Gray	2D Gray	2D RGB or YUV	N/A	2D/3D RGB-D

* Bold font indicates remarkable performance compared with other processors in this table.

For vision chips with image sensors and processors, the fabrication process, area, and voltage only consider the processor part (unless explicitly stated).

The work in [8] has 4 different supply voltages (3.3/2.9/1.8/1.1 V) but not mentioned which is for the processor part.

*¹ This work is an independent vision SoC receiving VGA like image.

*² Scaled to 1.08 V for approximate estimation, scaling formula is $1/U^2$ (U = desired voltage / starting voltage) [41].

*³ Scaled to 55 nm for approximate estimation, scaling formula is $1/S^2$ (S = desired node / starting node)[41].

*⁴ Only output inference result. If output 1920×1080 image from high-speed TX, the measured power is 356 mW.

*⁵ Only compute core area without SRAM.

*⁶ Tested result on MoblieNet V1.

*⁷ The result is measured with 80% weight sparsity & 50% activation sparsity at 0.5 V & 16 MHz.

*⁸ Measured at 5 MHz, 0.46 V, 16b. The effective energy efficiency is 2.1 TOPS/W with an approximation method.

*⁹ Measured at the max clock frequency.

is cut from the full 1920 × 1080 input image, and the background position relative to the full image depends on the target position in the previous frame. The Quantized algorithm parameters of SiamFC-Mobile were reduced to 1 MB in total. The test system achieved 27 fps with input images from FPGA. The processing speed of the visual processor reached 36 fps. Fig.15 shows the inference results on a subset of the OTB100 dataset [39] and CNN model structure. The target in the background can be captured by SiamFC-Mobile even if the target is partially covered by an uninterested object.

C. Comparison With Previous Work

Table.IV compares the performance of the proposed visual vector processor with some previous state-of-the-art programmable vision processors. The proposed visual processor achieves the highest flexibility for 2D/3D multimodal processing through instruction-level programmability and corresponding implementation methods. The programmable PE array of our processor integrates hybrid compute units to support micro-operations of multimodal algorithms. After scaling to the same technology (55 nm) for an estimation (use the scaling method in [41]), our processor achieves the highest area efficiency among vision chip processors in Table.IV. Besides, our work does not require extra ISP. Through flexible 2D/3D compatible ISA, all the image processing tasks are executed by the proposed PE array. Therefore, the large-area application-specific image processor is eliminated to save chip area. The vision chip of [8] is a 3D stacked vision chip

with high resolution, but the 4-bit PE is not appropriate for high-accuracy vision tasks. The work of [13] only achieves relatively high performance at a low frequency. The work of [14] is a 3D stacked vision chip dedicated to conventional CV with 5500 fps, yet it lacks a MAC unit to accelerate CNN. The work of [15] is a 3D stacked vision chip integrating a high-resolution image sensor and a CNN processor sub-system. Though it achieves the highest performance on CNN in Table.IV, the area consumption of the sub-system is large due to the separate regions used for specific tasks. The work of [40] is a vision processor integrating low-power embedded RAM and systolic PE array, but it ignores the compatibility with a high-speed image sensor. It results in severe limitations in many edge applications that require data processing close to the source. It's 2D-Mapping [40] architecture applies a multi-cycle single-weight broadcast dataflow which is only efficient for 2-D filter computing. Compared with the prior vision processors that support both CNN and non-CNN tasks [18], [40] our proposed ISA supports flexible parallel PE micro-operations for various 2D/3D image processing. The architecture of our PE is complex when integrating the proposed ISA, therefore we implement 256 PEs in the processor. Though the peak energy efficiency of the proposed visual vector processor in this paper is limited by low PE number and large on-chip SRAM, it still achieves competitive performance with the ISA and mapping scheme we design. Though we use a relatively backward technology compared with [15] and [40], the maximum operating frequency of our fabricated chip is competitive

due to the design of the ISA pipeline. The vectorial programmable PE array allowed for a trade-off between flexibility and performance compared with dedicated computing edge device [42]. Recently, processing-in-sensor (PIS) vision chip [43], [44], [45] has achieved ultra-low power consumption by utilizing photo-currents or signal-dependent pulses for computing. The work of [43] achieves power consumption of $80.4 \mu\text{W}$ with a low fill factor of 36% and 3-bit data precision. The challenge of tightly-coupled pixel-PE structure mentioned in section I remains in these PIS vision chips.

D. Discussion

The main point of this paper is to exploit a general architecture for multimodal data processing vision chips. We believe the visual vector processor described in this paper is able to integrate with a multimodal 2D/3D image sensor (like the work in [26]) to form a complete vision chip architecture. All the computing tasks of raw data from the sensor can be executed on this architecture to realize fully-on-chip 2D/3D image capture, storage, and processing. The development of algorithms especially DNN models is much faster than edge devices, flexible architecture has the potential to tolerate new DNN models. Therefore, we design an instruction-level programmable PE array for flexible parallel processing with a flexible ISA. The visual vector processor also implements large SRAM for a localized solution without dependency on external memory with high data safety during processing. Though it results in low efficiency compared with dedicated SoCs, the proposed processor achieves high flexibility with competitive performance. A standard high-speed interface offers a solution for near-sensor computing to reduce the bottleneck of image read-in delay. The efficiency of this processor can be improved in two aspects. In the aspect of circuit design, the processor can reduce power consumption and total delay if the PE array has a higher reuse rate with a new reconfigure mode. In the aspect of algorithm optimization, a penalty-based structural pruning method for lightweight CNN in [46] can be applied to obtain further efficiency for faster on-chip inference.

VI. CONCLUSION

In this paper, we propose a programmable 2D/3D perception visual vector processor for a high-speed vision chip. The work in this paper is an edge computing SoC for multimodal image processing through the proposed 2D/3D compatible ISA. Reconfigurable PE array and dual-issue micro-architecture are integrated into this visual processor to deal with complex functions and high throughput. The processor supports general 2D/3D vision algorithms and optimizes performance, especially for CNN. The peak computing performance, peak energy efficiency, and area efficiency of the fabricated processor are 409.6 GOPS, 1.49 TOPS/W and 14.41 GOPS/mm^2 at 200 MHz. It can efficiently implement both CNN and Conventional 2D/3D image processing algorithms through 2D/3D compatible ISA and mapping schemes. Compared to other reported processors of vision chips, our design accomplished comparably high area efficiency and obtained real-time processing for 2D/3D CV tasks, such as depth reconstruction and

object detection. The processor has the capacity to integrate with a high-speed 2D/3D sensor, therefore demonstrating the solution for fully-on-chip intelligence.

REFERENCES

- [1] C. Mead, "Neuromorphic electronic systems," *Proc. IEEE*, vol. 78, no. 10, pp. 1629–1636, Jan. 1990.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, Feb. 2015.
- [3] M. Ishikawa, K. Ogawa, T. Komuro, and I. Ishii, "A CMOS vision chip with SIMD processing element array for 1 ms image processing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, 1999, pp. 206–207.
- [4] J. A. Schmitz, M. K. Gharzai, S. Balkir, M. W. Hoffman, D. J. White, and N. Schemm, "A 1000 frames/s vision chip using scalable pixel-neighborhood-level parallel processing," *IEEE J. Solid-State Circuits*, vol. 52, no. 2, pp. 556–568, Feb. 2017.
- [5] W. Miao, Q. Lin, W. Zhang, and N.-J. Wu, "A programmable SIMD vision chip for real-time vision applications," *IEEE J. Solid-State Circuits*, vol. 43, no. 6, pp. 1470–1479, Jun. 2008.
- [6] Q. Lin, W. Miao, W. Zhang, Q. Fu, and N. Wu, "A 1,000 frames/s programmable vision chip with variable resolution and row-pixel-mixed parallel image processors," *Sensors*, vol. 9, no. 8, pp. 5933–5951, Jul. 2009.
- [7] C.-C. Cheng, C.-H. Lin, C.-T. Li, and L.-G. Chen, "iVisual: An intelligent visual sensor SoC with 2790 fps CMOS image sensor and 205 GOPS/W vision processor," *IEEE J. Solid-State Circuits*, vol. 44, no. 1, pp. 127–135, Jan. 2009.
- [8] T. Yamazaki et al., "A 1 ms high-speed vision chip with 3D-stacked 140 GOPS column-parallel PEs for spatio-temporal image processing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2017, pp. 82–83.
- [9] W. Zhang, Q. Fu, and N.-J. Wu, "A programmable vision chip based on multiple levels of parallel processors," *IEEE J. Solid-State Circuits*, vol. 46, no. 9, pp. 2132–2147, Sep. 2011.
- [10] C. Shi et al., "A 1000 fps vision chip based on a dynamically reconfigurable hybrid architecture comprising a PE array processor and self-organizing map neural network," *IEEE J. Solid-State Circuits*, vol. 49, no. 9, pp. 2067–2082, Sep. 2014.
- [11] J. Yang, Y. Yang, Z. Chen, L. Liu, J. Liu, and N. Wu, "A heterogeneous parallel processor for high-speed vision chip," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 28, no. 3, pp. 746–758, Mar. 2018.
- [12] J. Guo et al., "A vision chip architecture for image recognition based on improved convolutional neural network," in *Proc. IEEE Int. Conf. Electron Devices Solid-State Circuits (EDSSC)*, Aug. 2016, pp. 310–313.
- [13] K. Bong, S. Choi, C. Kim, D. Han, and H.-J. Yoo, "A low-power convolutional neural network face recognition processor and a CIS integrated with always-on face detector," *IEEE J. Solid-State Circuits*, vol. 53, no. 1, pp. 115–123, Jan. 2018.
- [14] L. Millet et al., "A 5500-frames/s 85-GOPS/W 3-D stacked BSI vision chip based on parallel in-focal-plane acquisition and processing," *IEEE J. Solid-State Circuits*, vol. 54, no. 4, pp. 1096–1105, Apr. 2019.
- [15] R. Eki et al., "A 1/2.3 inch 12.3 Mpixel with on-chip 4.97 TOPS/W CNN processor back-illuminated stacked CMOS image sensor," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 154–156.
- [16] M. Lepecq, T. Dalgaty, W. Fabre, and S. Chevobbe, "End-to-end implementation of a convolutional neural network on a 3D-integrated image sensor with macropixel array," *Sensors*, vol. 23, no. 4, p. 1909, Feb. 2023.
- [17] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. Imag. Understand. Workshop*, 1981, pp. 121–130.
- [18] X. Zheng et al., "ViP: A hierarchical parallel vision processor for hybrid vision chip," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 6, pp. 2957–2961, Jun. 2022.
- [19] Q. Luo et al., "A programmable and flexible visual processor," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 9, pp. 3884–3888, Jan. 2022.
- [20] O. Kumagai et al., "A 189×600 back-illuminated stacked SPAD direct time-of-flight depth sensor for automotive LiDAR systems," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 110–112.

- [21] K. I. Withanage, I. Lee, R. Brinkworth, S. Mackintosh, and D. Thewlis, "Fall recovery subactivity recognition with RGB-D cameras," *IEEE Trans. Ind. Inform.*, vol. 12, no. 6, pp. 2312–2320, Dec. 2016.
- [22] G. Kim et al., "A 1.22 tops and 1.52 mW/MHz augmented reality multicore processor with neural network NoC for HMD applications," *IEEE J. Solid-State Circuits*, vol. 50, no. 1, pp. 113–124, Jan. 2015.
- [23] C.-H. Chan et al., "Trending IC design directions in 2022," *J. Semiconductors*, vol. 43, no. 7, Jul. 2022, Art. no. 071401.
- [24] S.-J. Kim, B. Kang, J. D. K. Kim, K. Lee, C.-Y. Kim, and K. Kim, "A 1920×1080 3.65 μm -pixel 2D/3D image sensor with split and binning pixel structure in 0.11 μm standard CMOS," in *Proc. IEEE Int. Solid-State Circuits Conf.*, Feb. 2012, pp. 396–398.
- [25] H. Huang, K. Wei, and Y. Zhao, "A compound optofluidic lens for switchable 2D/3D imaging," in *Proc. IEEE 29th Int. Conf. Micro Electro Mech. Syst. (MEMS)*, Jan. 2016, pp. 2–5.
- [26] Z. Chen et al., "A 256×256 time-of-flight image sensor based on center-tap demodulation pixel structure," *Sci. China Inf. Sci.*, vol. 59, no. 4, pp. 1–10, Apr. 2016.
- [27] M. Nagel, M. Fournarakis, R. Ali Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," 2021, *arXiv:2106.08295*.
- [28] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.
- [29] G. Zach, M. Davidovic, and H. Zimmermann, "A 16 × 16 pixel distance sensor with in-pixel circuitry that tolerates 150 klx of ambient light," *IEEE J. Solid-State Circuits*, vol. 45, no. 7, pp. 1345–1353, Jul. 2010.
- [30] M. Xu, M. Zhao, S. Yu, X. Zheng, N. Wu, and L. Liu, "Rounding shift channel post-training quantization using layer search," in *Proc. 4th Int. Conf. Artif. Intell. Big Data (ICAIBD)*, May 2021, pp. 545–549.
- [31] X. Zheng, M. Zhao, Q. Luo, S. Yu, L. Liu, and N. Wu, "A chip-level verification method for programmable vision chip based on deep learning algorithms," in *Proc. IEEE 5th Int. Conf. Integr. Circuits Microsystems (ICIM)*, Oct. 2020, pp. 281–284.
- [32] M. Zhao et al., "A verification method for array-based vision chip using a fixed-point neural network simulation tool," in *Proc. IEEE 11th Latin Amer. Symp. Circuits Syst. (LASCAS)*, Feb. 2020, pp. 1–4.
- [33] P. S. Huang, "Novel method for structured light system calibration," *Opt. Eng.*, vol. 45, no. 8, Aug. 2006, Art. no. 083601.
- [34] K. Lai, L. Bo, X. Ren, and D. Fox, "A large-scale hierarchical multi-view RGB-D object dataset," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2011, pp. 1817–1824.
- [35] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.
- [36] W. Liu et al., "SSD: Single shot MultiBox detector," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 21–37.
- [37] S. Yang, P. Luo, C. C. Loy, and X. Tang, "WIDER FACE: A face detection benchmark," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 5525–5533.
- [38] M. Cen and C. Jung, "Fully convolutional Siamese fusion networks for object tracking," in *Proc. 25th IEEE Int. Conf. Image Process. (ICIP)*, Oct. 2018, pp. 3718–3722.
- [39] Y. Wu, J. Lim, and M.-H. Yang, "Online object tracking: A benchmark," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2013, pp. 2411–2418.
- [40] Q. Zhang et al., "A 22 nm 3.5 TOPS/W flexible micro-robotic vision SoC with 2 MB eMRAM for fully-on-chip intelligence," in *Proc. IEEE Symp. VLSI Technol. Circuits*, Jun. 2022, pp. 72–73.
- [41] J. M. Rabaey, *Digital Integrated Circuits a Design Perspective*. USA: Pearson Education, 2003.
- [42] S. Kim, S. Kim, J. Lee, and H.-J. Yoo, "A low-power graph convolutional network processor with sparse grouping for 3D point cloud semantic segmentation in mobile devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 4, pp. 1507–1518, Apr. 2022.
- [43] H. Xu et al., "Senputing: An ultra-low-power always-on vision perception chip featuring the deep fusion of sensing and computing," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 1, pp. 232–243, Jan. 2022.
- [44] M. Lefebvre, L. Moreau, R. Dekimpe, and D. Bol, "A 0.2-to-3.6 TOPS/W programmable convolutional imager SoC with in-sensor current-domain ternary-weighted MAC operations for feature extraction and region-of-interest detection," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 118–120.
- [45] T.-H. Hsu et al., "A 0.8 V intelligent vision sensor with tiny convolutional neural network and programmable weights using mixed-mode processing-in-sensor technique for image classification," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.
- [46] M. Zhao, J. Peng, S. Yu, L. Liu, and N. Wu, "Exploring structural sparsity in CNN via selective penalty," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 3, pp. 1658–1666, Mar. 2022.



Siyuan Wei received the B.S. degree in electronic science and technology from the University of Chinese Academy of Sciences (CAS), Beijing, China, in 2020. He is currently pursuing the Ph.D. degree with the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, CAS, Beijing. His research interests include vision chips, microprocessor chips, CNN processors, and hardware/software co-design.



Ke Ning received the B.S. degree in electronic science and technology from the Huazhong University of Science and Technology, Wuhan, China, in 2016, and the Ph.D. degree from the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), Beijing, China, in 2022. His research interests include vision chips, computer vision, microprocessor chips, and hardware/software co-design.



Lei Kang received the B.S. degree in materials science and optoelectronic technology from the University of Chinese Academy of Sciences (CAS), Beijing, China, in 2018, and the Ph.D. degree from the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, CAS, Beijing, in 2023. His research interests include vision chips, sparse methods, dynamic vision sensors, and spiking neural network accelerators.



Xuemin Zheng received the B.S. degree in applied physics from the University of Science and Technology of China, Hefei, China, in 2016, and the Ph.D. degree from the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), Beijing, China, in 2022. His research interests include vision chips, computer vision, microprocessor chips, deep learning, and compilers.



Mingxin Zhao received the B.S. degree in applied physics from the University of Science and Technology of China, Hefei, China, in 2017, and the Ph.D. degree from the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), Beijing, China, in 2022. His research interests include neural network compression and acceleration, neural network accelerator, and hardware/software co-design.



Mengmeng Xu received the B.S. degree from Zhengzhou University, Henan, China, in 2018, and the Ph.D. degree from the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), Beijing, China, in 2023. Her research interests include deep learning, neural network accelerator, hardware/software co-design, and neural network quantization.



Shuyu Wang received the B.S. degree from the Beijing University of Posts and Telecommunications, China, in 2018, and the master's degree from the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), Beijing, China, in 2021. Her research interests include computer vision, deep learning, and image pre-processing methods.



Xuanzhe Xu received the B.S. degree in electronic science and technology from the University of Chinese Academy of Sciences (CAS), Beijing, China, in 2021. He is currently pursuing the Ph.D. degree with the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, CAS, Beijing. His research interests include vision chips, radar signal processing, and microprocessors.



Runjiang Dou (Member, IEEE) received the B.Sc. and M.Sc. degrees in control science and engineering: pattern recognition from the Beijing Institute of Technology, China, in 2008 and 2010, respectively, and the Ph.D. degree in microelectronics and solid state electronics from the University of Chinese Academy of Sciences (CAS), Beijing, China, in June 2020. Since 2011, he has been with the Institute of Semiconductors, CAS, China, where he is currently a Senior Engineer. His current research interests include designing vision system based on intelligent hardware.



Shuangming Yu received the B.S. degree in electronic science and technology from the Beijing University of Posts and Telecommunications, China, in 2010, and the Ph.D. degree in microelectronics and solid-state electronics from the Institute of Semiconductors, Chinese Academy of Sciences (CAS), China, in 2015. He is currently an Associate Professor with the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, CAS. His current research interests include vision chip design and ultra-low power circuit design.



Xu Yang (Member, IEEE) received the B.S. degree from the Beijing Institute of Technology, Beijing, China, in 2015, and the Ph.D. degree in microelectronics and solid-state from the University of Chinese Academy of Sciences (CAS), Beijing, in 2020. In 2023, she finished her Post-Doctoral Fellow training with the Institute of Semiconductors, CAS, Beijing, where she is currently a Research Assistant. Her current research interests include bio-inspired spiking vision chips, spiking neural network and image processing algorithms, and parallel image processor architectures.



Jian Liu received the B.S. degree in semiconductor physics and devices from Jilin University, Changchun, China, in 1988, the M.Phil. degree in physics from the University of Birmingham, Birmingham, U.K., in 1998, and the Dr. rer. nat. degree in physics from Julius-Maximilians-Universität Würzburg, Würzburg, Germany, in 2003. He is currently a Full Professor in microelectronics and solid state electronics with the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), China, and the Center of Materials Science and Optoelectronics Engineering, University of CAS, China. His current research interests include semiconductor optoelectronic detectors, THz imager, and metamaterials.



Cong Shi (Member, IEEE) received the Ph.D. degree in electrical engineering from Tsinghua University, Beijing, China, in 2014. He was a joint Ph.D. student with the Institute of Semiconductors, Chinese Academy of Sciences (CAS), China. From 2015 to 2018, he was a Post-Doctoral Fellow with the Schepens Eye Research Institute, Massachusetts Eye and Ear, Harvard Medical School, Boston, MA, USA. He is currently a Research Professor with the School of Microelectronics and Communication Engineering, Chongqing University, China, where he leads the research group in AI chip designs for smart visual processing and neuromorphic systems.



Nanjian Wu (Senior Member, IEEE) received the B.S. degree in physics from Heilongjiang University, China, in 1982, the M.S. degree in electronic engineering from Jilin University, Changchun, China, in 1985, and the D.S. degree in electronic engineering from The University of Electronic Communications, Tokyo, Japan, in 1992. In 1992, he joined the Research Center for Interface Quantum Electronics and the Faculty of Engineering, Hokkaido University, Sapporo, Japan, as a Research Associate. In 1998, he was an Associate Professor with the Department of Electro-Communications, The University of Electronic Communications. Since 2000, he has been a Professor with the Institute of Semiconductors, Chinese Academy of Sciences (CAS). In 2005, he was a Visiting Professor with the Research Center for Integrated Quantum Electronics, Hokkaido University. Since 2009, he has been an Honorable Guest Professor with the Research Institute of Electronics, Shizuoka University. His current research interests include mixed-signal LSI and vision chip design.



Liyuan Liu (Member, IEEE) received the B.S. and Ph.D. degrees in electrical engineering from the Electronic Engineering Department, Institute of Microelectronics, Tsinghua University, Beijing, China, in 2005 and 2010, respectively. In 2012, he joined the State Key Laboratory of Superlattices and Microstructures, Institute of Semiconductors, Chinese Academy of Sciences (CAS), Beijing, as an Associate Professor, where he became a Professor in 2018. His current research interests include high speed CMOS image sensors, SPAD-based 3-D image sensors, CMOS terahertz image sensors, smart processors for edge computing, and the design of monolithic vision chip. He is a member of Youth Innovation Promotion Association, CAS, and a Committee Member of the Beijing Institute of Electronics (BIE). He serves as the Vice Chair of IEEE SSCS Beijing Chapter. He was a TPC Member for the IEEE Asian Solid-State Circuits Conference (A-SSCC) from 2013 to 2018. He was the TPC Chair for IEEE International Conference on Integrated Circuits Technologies and Applications (ICTA) from 2019 to 2020. He is an Associate Editor of *Journal of Semiconductors*.